

内容理解ノート

楽譜データ・4声輪唱・ドラム伴奏・音色再現

作成者：齋藤 翔太 作成日：2026年6月24日 更新日：2026年7月7日

1. この資料の目的

これは発表用のスライドではなく、担当したプログラムがどのように音を鳴らしているかを事前に理解するための資料です。当初はweek9の「かえるのうた」4声輪唱確認用Processingスケッチを対象に作成しました。現在は、week10で作成した修正版スケッチと、week11で整理したフルート・オルガン追加後の構成も反映しています。

理解する順番

- ・ 楽譜をどの形式で保持しているか
- ・ 同じ楽譜を4つの金管パートでどう使い分けているか
- ・ ドラムがどの拍で、どのように鳴るか
- ・ 旋律楽器とドラムの音色を、どのパラメータから再現しているか
- ・ week9版からweek10調整版へ、どこが変わったか

2. 全体の流れ

スケッチは、楽譜データを読み、各イベントを再生時刻に並べ、最後に音色定義を使って音を作ります。旋律4声とドラムでは、同じ再生スケジュールの仕組みを使いますが、音程の扱いと音色の選び方が異なります。

楽譜データ → パートごとの開始拍・音域設定 → 再生予約 → 音色JSONを使った発音

データ	役割
ScoreEvent	いつ、どの音を、どのくらいの強さ・長さで鳴らすかを表す。
PartDefinition	楽器名、音色JSON、開始拍、オクターブ移調量、基本音量を表す。

3. 楽譜データの読み方

durationQ8は拍を256分割した値です。256なら1拍、128なら0.5拍です。subNote以降は、同じ拍の途中で追加で鳴らす音を表し、ドラムの裏拍ハイハットなどに使います。

項目	意味
beatAt	曲の先頭から数えた開始拍
noteNumber	MIDIノート番号。休符は0
velocity	音の強さ。0から127の範囲
durationQ8	音の長さ。256で1拍
subNote / subOffsetQ8	1つ目の音に重なる音と、その開始位置

主旋律はMELODY_SCOREに1回だけ記述します。4パートで別々の譜面を持たないため、旋律を直すときはこの共通譜を修正すれば十分です。

4. week9版の4声輪唱の仕組み

トランペットから順に8拍ずつ遅らせて入ります。チューバは低音伴奏ではなく、24拍遅れて入る4声目の主旋律として扱っています。

パート	開始拍	移調量	基本音量	主な音域
トランペット	0	+12半音	0.20	C5からA5
ホルン	8	0半音	0.17	C4からA4
トロンボーン	16	-12半音	0.18	C3からA3
チューバ	24	-24半音	0.15	C2からA2

再生時には、各イベントの開始時刻にパートの開始拍を足します。金管の音程は、「共通譜のnoteNumber + octaveShift」で決まります。これを周波数に変換して、金管の合成音を作るため、同じ旋律を高音から低音まで自然に配置できます。

5. 最新版の4声構成

week10の修正版では、発表前確認で聞き分けやすいように4声の担当楽器を変更しました。ホルンとチューバを外し、フルート、トランペット、トロンボーン、オルガンの構成にしています。

パート	開始拍	移調量	基本音量	主な音域
フルート	0	+12半音	0.28	C5からA5
トランペット	8	+12半音	0.22	C5からA5
トロンボーン	16	-12半音	0.22	C3からA3
オルガン	24	-12半音	0.25	C3からA3

フルートは倍音合成だけでは質感が弱くなりやすいため、音色JSON内のtone_sampleを使う原音サンプル主導の再生に変更しました。フルート用の補正値はFLUTE_SAMPLE_GAIN = 1.32です。全体音量はMASTER_GAIN = 1.35、ドラム音量はDRUM_AMPLITUDE = 0.095に調整しています。

この最新版でも、共通譜MELODY_SCOREを4パートで使い回す考え方は変えていません。変わったのは、PartDefinitionで指定する音色JSON、音域、音量です。

6. ドラム伴奏の仕組み

ドラム譜はcreateDrumScore()で生成し、曲の最後まで支えられるように56拍分を用意しています。最新版では4分の4拍子として、1・3拍目をキック、2・4拍目をスネアに整理しています。

拍	主音	追加音
1拍目	キック。小節の頭を示す。	裏拍にハイハット
2拍目	スネア。拍の区切りを示す。	裏拍にハイハット
3拍目	キック。低音の支えを作る。	裏拍にハイハット
4拍目	スネア。次の小節へつなぐ。	裏拍にハイハット

特別な処理

- ・ 0、8、16、24拍目：新しい声部が入る位置で、控えめなクラッシュとキックを同時に鳴らす。
- ・ 最後の拍：クラッシュとキックで終止感を出す。

最新版のドラム基本音量はDRUM_AMPLITUDE = 0.095です。各音では、さらにvelocity / 127を掛けます。旋律より目立ちすぎず、拍を感じられる音量にする意図です。

7. 音を再現するパラメータ

金管もドラムも、音色JSONにある倍音、ノイズ、ADSRエンベロープを読み取ります。最新版では、フルートのtone_sampleも読み取ります。ADSRは、音の立ち上がりから消えるまでの形を表します。

パラメータ	音への影響
fundamental_hz	基準となる周波数。ドラムでは音の中心的な高さを決める。
harmonics	倍音の周波数比と振幅。音の明るさや厚みを決める。
noise_level	ノイズ成分の量。スネアやシンバルのざらつきに関係する。
attack_sec	音が立ち上がるまでの時間。
decay_sec / sustain_level	立ち上がった後の減衰と、保持される音量。
release_sec	音を止めた後に消えるまでの時間。
tone_sample	フルートなどの原音本体サンプル。倍音合成より自然な質感を優先したい場合に使う。
drum_sample	解析済みの1打分の波形。ある場合は、合成音より優先して再生する。

week9で使用しているドラム音色の主な設定値

音色	基準Hz	倍音数	Attack	Decay	Release	Noise
キック	39.029	40	0.008	0.190	0.560	0.0139
スネア	180.341	40	0.005	0.320	0.260	0.0252
ハイハット	263.782	36	0.005	0.900	0.500	0
クラッシュ	429.318	32	0.012	0.900	1.500	0.0479

上の値は「どの楽器らしく聞こえるか」を決める値です。譜面のvelocityは、演奏中の音の強さを決める値であり、役割が異なります。

8. ドラムは旋律楽器と同じ方法か

土台となる処理は共通です。どちらも再生時刻を予約し、音色JSONを読み、音量と音の長さを反映して発音します。ただし、ドラムには専用の扱いがあります。

観点	旋律楽器	ドラム
音程	共通譜のノートをパートごとにオクターブ移調する。	キック等のノート番号から、対応する音色JSONを選ぶ。
音色	倍音を重ね、ADSRで形を作る。フルートはtone_sampleを優先する。	drum_sampleがあれば優先再生し、なければ倍音・ノイズ・ADSRで合成する。
停止処理	譜面上の長さでノートオフする。	原音波形を使う場合は、自然な減衰を残すため途中で止めない。

9. week9版から最新版への変更点

観点	week9版	最新版
旋律4声	トランペット、ホルン、トロンボーン、チューバ	フルート、トランペット、トロンボーン、オルガン
フルート	未使用	tone_sampleを使い、FLUTE_SAMPLE_GAIN = 1.32で再生
全体音量	控えめな確認用	MASTER_GAIN = 1.35で発表前確認向けに調整
ドラム音量	DRUM_AMPLITUDE = 0.075	DRUM_AMPLITUDE = 0.095
ドラムパターン	4拍パターンと終止前フィルを含む	4分の4拍子として整理し、最後の拍をクラッシュ + キックで強調
音色ファイル	金管4種類とドラム4種類	フルート、トランペット、トロンボーン、オルガン、ドラム4種類

最新版は、Arduino実機用の最終ファームウェアそのものではなく、発表前に旋律・音色・バランスを確認するためのProcessingスケッチです。そのため、音色JSONをスケッチ内のdataフォルダに同梱し、外部フォルダ参照なしで再生できるようにしています。

10. 確認済みのことと、事前に見ておくこと

確認済み

- ・ 最新版の4声の開始拍が0、8、16、24拍であること。
- ・ フルート、トランペット、トロンボーン、オルガンの4声構成になっていること。
- ・ フルート音色JSONにtone_sampleがあり、スケッチ側でToneSampleUGenを使うこと。
- ・ ドラム譜が56拍あり、声部の入りと最後の拍にクラッシュを含むこと。
- ・ 旋律4種類とドラム4種類、合計8個の音色JSONを読み込むこと。
- ・ Processingスケッチが起動時に例外を出さないこと。

本番前に確認すること

- ・ 実際に全パートを鳴らし、フルートとオルガンが他の旋律に埋もれていないかを聴く。
- ・ クラッシュ、ハイハット、最後の拍の強調が主旋律を邪魔していないかを聴く。
- ・ 必要ならMASTER_GAIN、PartDefinitionのamplitude、FLUTE_SAMPLE_GAIN、DRUM_AMPLITUDEを調整する。

11. ソースの対応箇所

実装を読み返すときは、次の順番が分かりやすいです。

順番	確認する箇所	何が分かるか
1	MELODY_PARTS と MELODY_SCORE	共通譜と、各旋律パートの時間差・音域
2	createDrumScore() と schedulePart()	ドラム譜の作り方と再生時刻の決め方
3	TimbreData、ToneSampleUGen、BrassNote、DrumNote、RecordedDrumNote	音色JSONと原音サンプルをどう使って音を作るか
4	data/ の *.tweaked.instrument.json	各楽器の具体的な音色パラメータ